FlexyPool comes with the following default failover strategies:

Table 2.3: FlexyPool failover strategies

Name	Description
Increment pool size on timeout	The connection pool has a *minimum size* and, on demand, it can grow up to its *maximum size*. This strategy increments the target connection pool maximum size on connection acquisition timeout. The *overflow* is a buffer of extra connections allowing the pool to grow beyond its initial *maximum size* until it reaches the *overflow size* threshold
Retrying attempts	This strategy is useful for those connection pools lacking a connection acquiring retry mechanism, and it simply reattempts to fetch a connection for a given number of tries

2.4.1 A real-life connection pool monitoring example

The following example demonstrates how FlexyPool failover strategies can determine the right connection pool size. The application under test is a batch processor using Bitronix transaction manager[5] as the database connection pooling provider.

The batch processor is given a certain data load, and the pool size automatically grows upon detecting a connection acquisition timeout occurrence. The average and the maximum pool size are determined experimentally, without the need of any prior mathematical calculations.

Prior to running the load testing experiment, it is better to know the current application connection pool settings. According to the Bitronix connection pool documentation[6] the default `acquisitionTimeout` (the maximum time a connection request waits before throwing a timeout exception) is 30 seconds.

A connection acquisition timeout threshold of one second is sufficient for the current experiment, allowing the application to react more quickly to a traffic spike and apply a compensating failover strategy.

The initial `maxPoolSize` is set to one connection, and, upon receiving a connection acquisition timeout, it grows until the `maxOverflow` threshold is reached.

The `retryAttempts` value is intentionally set to a reasonably large value because, for a batch processor, dropping a connection request is a much bigger problem than some occasional transaction response time spikes.

[5] https://github.com/bitronix/btm
[6] https://github.com/bitronix/btm/wiki/JDBC-pools-configuration

The experiment starts with the following initial connection pool settings:

Table 2.4: Initial connection pool settings

Name	Value	Description
minPoolSize	0	The pool starts with an initial size of 0
maxPoolSize	1	The pool starts with a maximum size of 1
acquisitionTimeout	1	A connection request waits for 1s before giving up with a timeout exception
maxOverflow	4	The pool can grow up to 5 connections (initial `maxPoolSize` + `maxOverflow`)
retryAttempts	30	If the final maxPoolSize is reached, and there is no connection available, a request will retry 30 times before giving up.

2.4.1.1 Concurrent connection request count metric

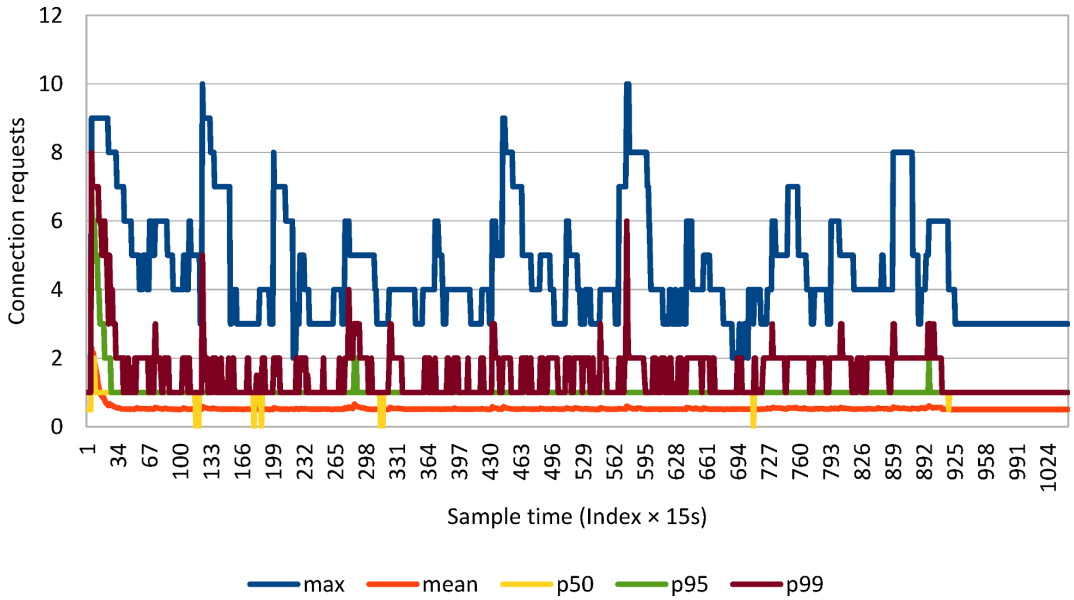

Figure 2.9: Concurrent connection requests

The more incoming concurrent connection requests, the higher the response time (for obtaining a pooled connection) gets. This graph shows the incoming request distribution, making it ideal for spotting traffic spikes.

The average value levels up all outliers, so it cannot reflect the application response to a given traffic spike.

When the recorded values fluctuate dramatically, the average and the maximum value alone offer only a limited view over the actual range of data, and that is why percentiles are preferred in application performance monitoring.

By offering the maximum value, relevant to only a percentage of the whole population, percentiles make outliers visible while capturing the immediate effect of a given traffic change.

2.4.1.2 Concurrent connection count metric

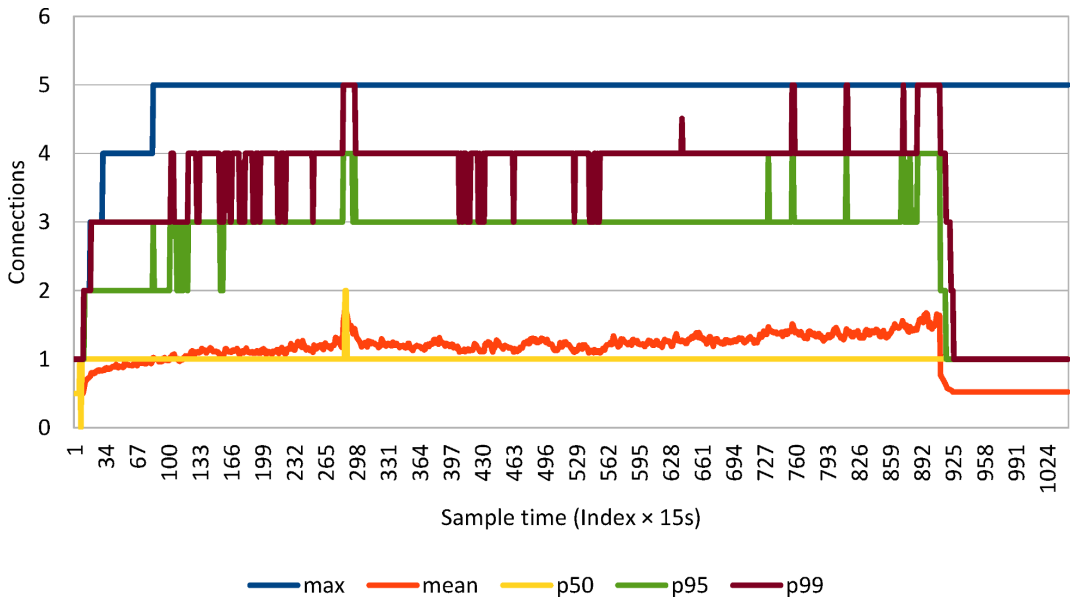

Figure 2.10: Concurrent connections

The average concurrent connection metric follows a gradual slope up to 1.5 connections. Unfortunately, this value is of little use for configuring the right pool size. On the other hand, the 99th percentile is much more informative, showing that 3 to 5 connections are sufficient. The maximum connections graph reconfirms that the pool size should be limited to 5 connections (in case the connection acquisition time is acceptable).

If the connection pool supports it, it is very important to set the idle connection timeout threshold. This way, the pool can release unused connections so the database can provide them to other clients as well.

2.4.1.3 Maximum pool size metric

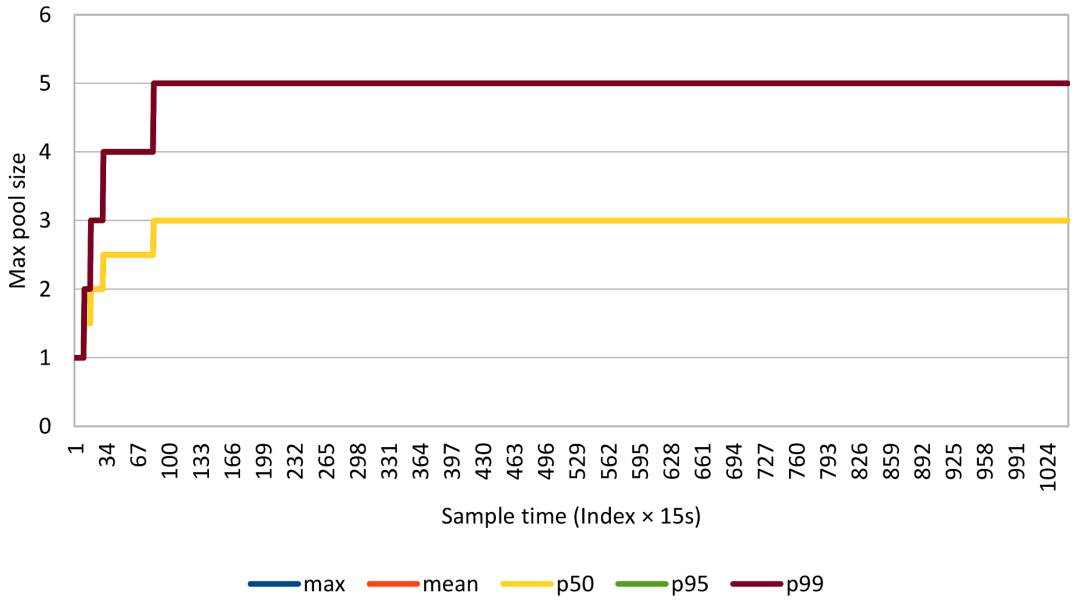

Figure 2.11: Maximum pool size

According to the 99th percentile, the pool gets saturated soon after the job process starts.

2.4.1.4 Connection acquisition time metric

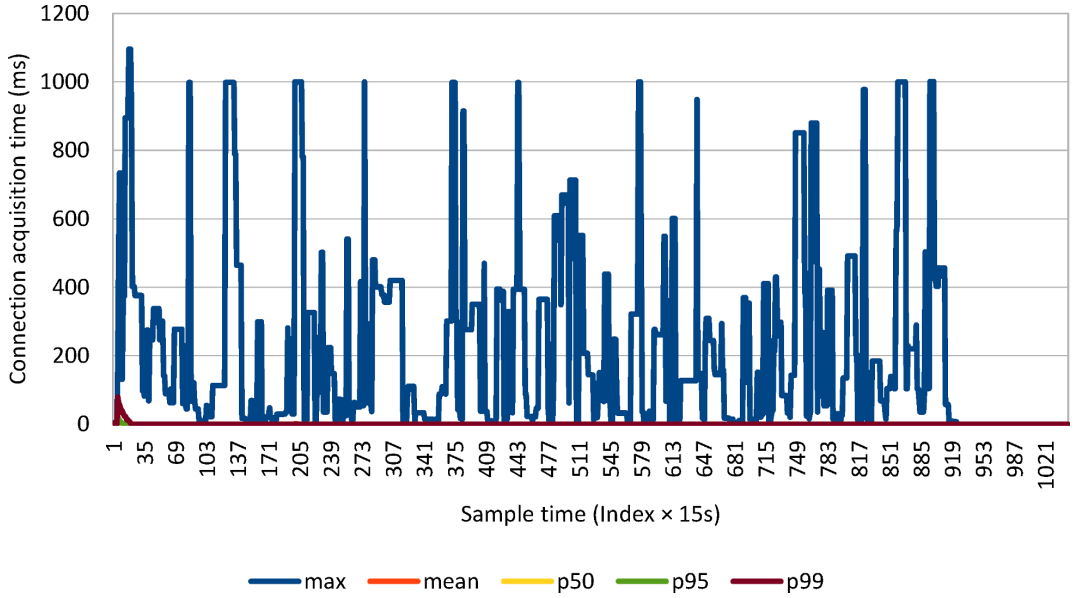

Figure 2.12: Connection acquisition time

The traffic spikes are captured by the maximum graph only. The timeout threshold is hit multiple times as the pool either grows its size or it retries the connection acquisition request.

2.4.1.5 Retry attempts metric

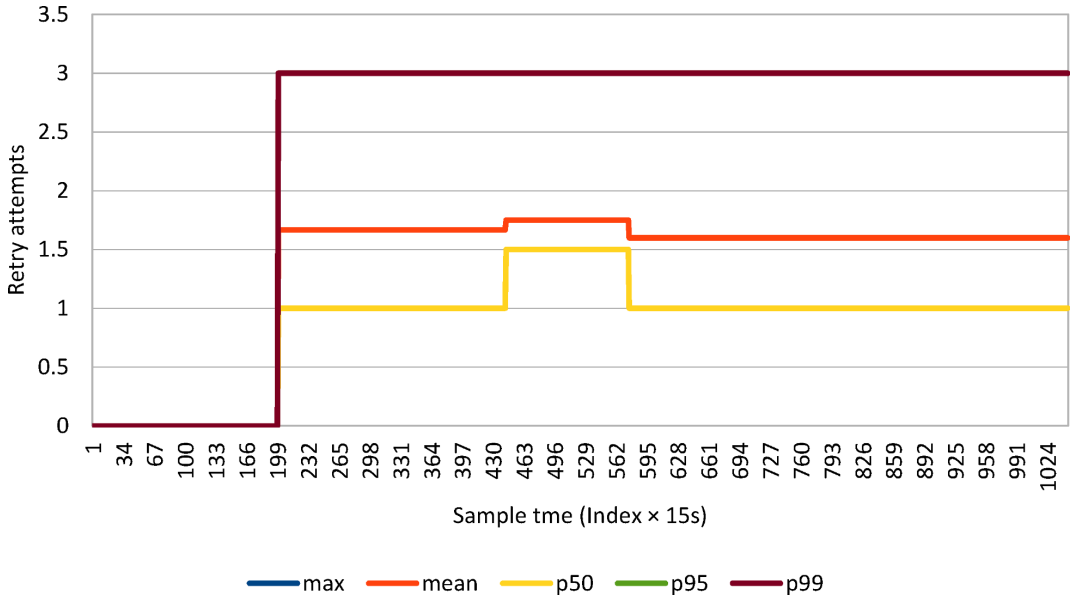

Figure 2.13: Retry attempts

When limiting the connection pool to 5 connections, there are only 3 retry attempts.

2.4.1.6 Overall connection acquisition time metric

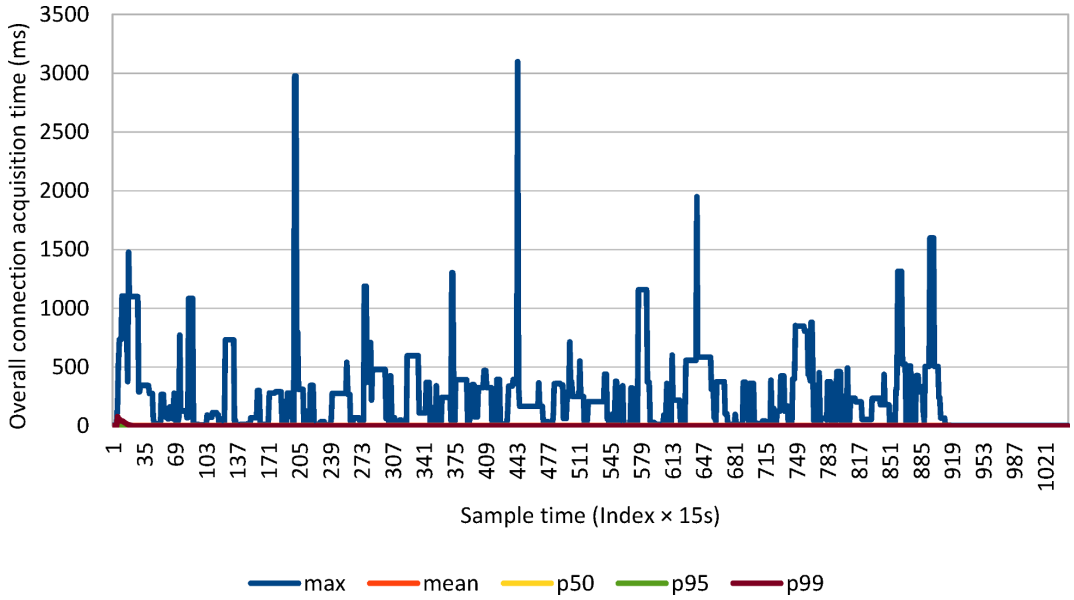

Figure 2.14: Overall connection acquisition time

While the retry attempts graph only shows how the retry count increases with time, the actual effect of reattempting is visible in the overall connection acquisition time.